

Reflection Template

Use Case	UC-12
Use Case Name	Change Task Status
Requirement IDs	FR13, FR14, FR19, FR20, FR22, FR23
Matching Feature Comparison Sets	CS-24, CS-25
Implementation Order	Kotlin first -> Java second

Feature Relevance

Feature/s:

Null safety, type inference

Did the feature/s occur naturally in this/these context/s?

Yes, (the status validation logic in the enum used when in Kotlin and switch in Java -> both as expressions.) Type inference via var was used naturally in the service. No features were forced.

Qualitative Ratings (1-5)

- 1 = very poor/very difficult
- 2 = rather poor/difficult
- 3 = neutral/moderate
- 4 = good/easy
- 5 = very good/very easy

Criteria	Java	Kotlin
Readability	5	4
Compactness of logic	5	5
Perceived implementation effort	5	5

Justifications

Readability:

Both are readable. Java's switch expression felt slightly more familiar than Kotlin's when this is a general language syntax difference, not directly related to the four investigated features. Both are functionally equivalent and equally compact.

Compactness of logic:

Both are equally compact. The enum method, service logic, and request DTO are all minimal in both languages.

Implementation effort:

Both felt equally straightforward. Nothing new.

Observed structural differences

Status validation is placed directly in the enum in both languages canTransitionTo method.

Kotlin's "when" must be exhaustive, the compiler forces you to handle all enum values. Java's switch expression also requires exhaustiveness in this context. No practical difference.

Both request DTOs are single-line, no meaningful difference.

Final comparative summary

UC12 is the first use case where Java rated higher on readability than Kotlin driven by a subjective preference for switch over when. Otherwise both implementations are equivalent in compactness and effort. The status validation logic is cleanly placed in the enum in both languages.